

Lesson 3 Reference: Output & Strings

Everything from this lesson on one page. Keep it open while you code.

Quick translation table

You want to...	JavaScript	Python
Print something	<code>console.log("Hi")</code>	<code>print("Hi")</code>
Print several items	<code>console.log("a", "b")</code>	<code>print("a", "b")</code>
Insert a variable	<code>`Hi \${name}`</code>	<code>f"Hi {name}"</code>
Repeat text	<code>"*".repeat(20)</code>	<code>"*" * 20</code>
End the statement	<code>;</code>	nothing
Comment	<code>// note</code>	<code># note</code>

print()

```
print("Hello, World!")           # Hello, World!
print("Welcome", "to", "Python") # Welcome to Python
print("Sum is", 7 + 3)           # Sum is 10
```

Two things to notice:

- **Commas add a space for you.** You don't type it.
- **Math is done first, then printed.** `7 + 3` becomes `10` before it appears.

No semicolons. They're legal in Python but PEP 8 forbids them, and style is part of your grade. Ruff will underline them.

sep and end

These two parameters are things `console.log()` can't do at all.

sep — what goes *between* the items

```
print("a", "b", "c")           # a b c      (default: one space)
print("a", "b", "c", sep="-")  # a-b-c
print("a", "b", "c", sep="")   # abc
print("a", "b", "c", sep=" > ") # a > b > c
```

end — what goes *after* everything

```
print("Loading")                # ends with a newline (the default)
print("Done")
# Loading
# Done

print("Loading", end="")        # ends with nothing
print("...done")
# Loading...done
```

Use `end=""` when you want several prints to land on one line.

Repeating text with *

```
print("-" * 40)                # 40 dashes
print("Hi! " * 3)              # Hi! Hi! Hi!
print(20 * "*")               # order doesn't matter
```

A string times a **number** repeats it.

```
print("*" * "20")              # TypeError: can't multiply sequence by non-int
```

You can never multiply a string by another string. This matters more than it looks — `input()` always hands you a *string*, so `"*" * input()` will fail for the same reason later on.

Building a box

The borders must match the width of the middle line:

```
print("=" * 10)
print("= PYTHON =")  # this line is 10 characters
print("=" * 10)
```

Quotes inside strings

Three approaches, in the order you should reach for them.

1. Switch the outer quotes — cleanest

```
print('She said "nice play"')  # double inside single
print("It's Bergen Tech's best team")  # single inside double
```

2. Escape with a backslash — works, harder to read

```
print('Display \'hi\' in quotes')
print("Display \"hi\" in quotes")
```

3. Triple quotes — when you need both kinds

```
print("""He said "that's mine" """)
```

Why does this break?

```
print('Display 'hi' in quotes')  # SyntaxError
```

Python thinks the string ends at the second `'`. Everything after it is nonsense to the interpreter.

f-strings

The direct equivalent of a JavaScript template literal.

```
name = "Alex"
age = 15

print(f"Hi {name}, you are {age}")    # Hi Alex, you are 15
```

Two differences from JS:

- Backticks become **normal quotes with an `f` in front**
- `${ }` becomes just `{ }`

The mistake everyone makes

```
print("Hi {name}")    # Hi {name}    <- forgot the f
print(f"Hi {name}")    # Hi Alex
```

No error appears. It just prints the braces. If you see `{ }` in your output, you left off an `f`.

Expressions work inside the braces

```
age = 15
price = 24.99
qty = 3

print(f"Next year you turn {age + 1}")    # Next year you turn 16
print(f"Total: {price * qty}")            # Total: 74.97
print(f"Loud: {name.upper()}")            # Loud: ALEX
```

If you need quotes *inside* the braces, use the other kind:

```
f"Item: {'Headphones'.upper()}"
```

Escape sequences

Identical to JavaScript.

Sequence	Meaning
<code>\n</code>	New line
<code>\t</code>	Tab
<code>\'</code>	A single quote
<code>\"</code>	A double quote
<code>\\</code>	One real backslash

```
print("Welcome\n\tto\n\nPython!")
# Welcome
# to
#
# Python!

print("Name\tAge\tGrade")          # Name    Age    Grade
print("Path: C:\\Users\\Student") # Path: C:\Users\Student
```

Two `\n` in a row produce a blank line. Two backslashes produce one — the first escapes the second.

Triple quotes

They do **one** job: let a string span multiple lines and hold both quote types.

```
message = '''Roses are red
Python is fun
Both quote types " and ' are fine in here'''

print(message)
```

Every line break you type is kept in the output. `'''` and `"""` work identically.

Triple quotes are NOT template literals

This is the single most common misunderstanding coming from JavaScript.

```
name = "Alex"
```

```
print("""Hi {name}""")    # Hi {name}    <- prints the braces  
print(f"""Hi {name}""")  # Hi Alex      <- the f does the work
```

	Multi-line?	Substitutes variables?
<code>"""..."""</code>	Yes	No
<code>f"..."</code>	No	Yes
<code>f"""..."""</code>	Yes	Yes

In JavaScript, backticks do both jobs at once. In Python they're separate tools — combine them with `f"""` when you need both.

Splitting a long line

```
message = ("This string is long, so we "  
           "split it across two lines")
```

Two strings side by side inside parentheses join automatically.

You may see `\` used at the end of a line for this. Avoid it — a single invisible space after the backslash breaks it, and PEP 8 prefers parentheses.

One you'll see in old code

```
print("Hi {}".format(name))  # the old way  
print(f"Hi {name}")          # the way we use
```

`.format()` came before f-strings. Recognize it in tutorials and older answers online; don't write it.

Common errors

Error or symptom	Cause	Fix
<code>SyntaxError</code> on a quoted line	Same quote type inside and outside	Switch the outer quotes or escape
Output shows <code>{name}</code> literally	Missing the <code>f</code> prefix	Add <code>f</code> before the opening quote
<code>TypeError: can't multiply sequence by non-int</code>	Multiplied a string by a string	The right side must be a number
<code>NameError: name 'x' is not defined</code>	Used a variable you never created, or a typo	Check spelling and that it's assigned above
Box or border doesn't line up	Middle line is a different width than the border	Count characters, or use <code>f"{text:<30}"</code>
Your changes don't appear	File wasn't saved	Ctrl/Cmd + S, then run again

Checklist before you push

- ☐ Header comment at the top: your name, assignment, date
- ☐ No semicolons anywhere
- ☐ `snake_case` variable names, not `camelCase`
- ☐ The file **runs** without errors
- ☐ Output matches the expected output exactly, including spaces
- ☐ Committed **and pushed** — check github.com to confirm